

---

# Asymptotic Analysis for Time and Space Complexity

Md. Owaliur Rahman

Department of CSE, Presidency University

September 20, 2026

# Outline

---

Runtime Function from Code

Asymptotic Analysis from Runtime Function

Space Complexity

Exam Practice

---

1

Runtime Function from Code

# 1, 2, 3

---

```
1 for (int i = 0; i < n; i++) {  
2     printf("%d\n", i);  
3 }
```

```
1 for (int i = 0; i < n; i += 2) {  
2     printf("%d\n", i);  
3 }
```

```
1 for (int i = n; i > 0; i /= 2) {  
2     printf("%d\n", i);  
3 }
```

## 4, 5

---

```
1  for (int i = 0; i < n; i++) {  
2      for (int j = 0; j < n; j++) {  
3          printf("%d %d\n", i, j);  
4      }  
5  }
```

```
1  for (int i = 0; i < n; i++)  
2      for (int j = 0; j < n; j++)  
3          for (int k = 0; k < n; k++) {  
4              s = i + j + k;  
5              printf("%d %d %d\n", s);  
6          }
```

```
1  for (int i = 0; i < n; i++) {  
2      int x = n;  
3      while (x > 1) {  
4          x = x / 2;  
5      }  
6  }
```

## 7, 8

---

```
1  int fibonacci(int n) {  
2      if (n == 0) return 0;  
3      if (n == 1) return 1;  
4      return fibonacci(n - 1) + fibonacci(n - 2);  
5  }
```

```
1  void f(int n) {  
2      if (n == 0) return;  
3      f(n - 1);  
4      f(n - 1);  
5      f(n - 1);  
6  }
```

```
1 void permute(int arr[], int n) {  
2     if (n == 1) return;  
3     for (int i = 0; i < n; i++) {  
4         permute(arr, n - 1);  
5     }  
6 }
```

# 10, 11

---

```
1  int sum(int a, int b) {  
2      return a + b;  
3  }
```

```
1  int triple(int a) {  
2      int out = a;  
3      out += a;  
4      out += a;  
5      return out;  
6  }
```

# Growth rates at a glance

---

| Growth       | Simplest pattern                                        | Complexity |
|--------------|---------------------------------------------------------|------------|
| Constant     | Fixed number of operations (e.g. <code>arr[0]</code> )  | 1          |
| Logarithmic  | Halve the problem each step (e.g. <code>i /= 2</code> ) | $\log n$   |
| Linear       | Single loop over $n$ elements                           | $n$        |
| Linearithmic | Nested $\log n$ loop inside an $n$ loop                 | $n \log n$ |
| Quadratic    | Two nested loops over $n$                               | $n^2$      |
| Cubic        | Three nested loops over $n$                             | $n^3$      |
| Exponential  | Two recursive calls per level                           | $2^n$      |
| Factorial    | Generate all permutations of $n$ items                  | $n!$       |

```
1  #define R1 2 // number of rows in Matrix-1
2  #define C1 2 // number of columns in Matrix-1
3  #define R2 2 // number of rows in Matrix-2
4  #define C2 3 // number of columns in Matrix-2
5
6  void multiplyMatrix(int m1[][C1], int m2[][C2])
7  {
8      int result[R1][C2];
9
10     for (int i = 0; i < R1; i++) {
11         for (int j = 0; j < C2; j++) {
12             result[i][j] = 0;
13
14             for (int k = 0; k < R2; k++)
15                 result[i][j] += m1[i][k] * m2[k][j];
16
17             printf("%d\t", result[i][j]);
18         }
19
20         printf("\n");
21     }
22 }
```

```
1  int binarySearch(int arr[], int n, int x) {  
2      int low = 0;  
3      int high = n-1;  
4      while (low <= high) {  
5          int mid = low + (high - low) / 2;  
6          if (arr[mid] == x) return mid;  
7          if (arr[mid] < x) low = mid + 1;  
8          else high = mid - 1;  
9      }  
10     return -1;  
11 }
```



# 2 Asymptotic Analysis from Runtime Function

# Definition

---

Asymptotic analysis is a method used in computer science to describe the **limiting behavior** of an algorithm's resource usage (typically time or space) as the input size  $n$  approaches infinity.

Instead of calculating the exact number of operations, it focuses on **how the running time or space requirements grow** with larger inputs.

This is expressed using asymptotic notations:

- **Big-O ( $O$ )** – upper bound (worst-case growth rate)
- **Big-Omega ( $\Omega$ )** – lower bound (best-case growth rate)
- **Big-Theta ( $\Theta$ )** – tight bound (exact growth rate)

# Example

---

$$T(n) = 3n^2 + 2n + 5$$

For large values of  $n$ , the term  $n^2$  dominates. The linear and constant terms become insignificant.

- **Big-O:**  $T(n) = O(n^2)$  or  $O(n^3)$  but **not**  $O(n)$  or  $O(n \log n)$
- **Big-Omega:**  $T(n) = \Omega(n^2)$  or  $\Omega(n)$  but **not**  $\Omega(n^3)$  or  $\Omega(2^n)$
- **Big-Theta:**  $T(n) = \Theta(n^2)$

# Tricky Example

---

$$T(n) = 2^n + n^5 + 100n^2$$

Even though  $n^5$  grows fast, the exponential term  $2^n$  dominates for large  $n$ . Polynomials grow much slower than exponentials.

- **Big-O** (Upper Bound):  $T(n) = O(2^n)$  or  $O(3^n)$  but **not**  $O(n^5)$  or  $O(n^{10})$
- **Big-Omega** (Lower Bound):  $T(n) = \Omega(2^n)$  or  $\Omega(n^5)$  but **not**  $\Omega(3^n)$  or  $\Omega(n!)$
- **Big-Theta** (Tight Bound):  $T(n) = \Theta(2^n)$

# Order of Growth

---

|          |   |             |   |        |   |              |   |           |   |       |   |             |   |           |
|----------|---|-------------|---|--------|---|--------------|---|-----------|---|-------|---|-------------|---|-----------|
| 1        | < | $\log n$    | < | $n$    | < | $n \log n$   | < | $n^2$     | < | $n^3$ | < | $2^n$       | < | $n!$      |
| Constant |   | Logarithmic |   | Linear |   | Linearithmic |   | Quadratic |   | Cubic |   | Exponential |   | Factorial |



3

Space Complexity

# Definition

---

Space complexity measures the **total memory** an algorithm needs as a function of the input size  $n$ .

Unlike time, we count **memory cells** instead of operations. The total splits into two parts:

- **Input space** – memory occupied by the input itself
- **Auxiliary space** – extra memory used while running (variables, arrays, recursion stack)

$$S(n) = \text{input space} + \text{auxiliary space}$$

Since the input size is fixed by the problem, we usually analyze the **auxiliary space**, using the same notations:  $O$ ,  $\Omega$ ,  $\Theta$ .

# What Counts as Space?

---

- **Fixed part** – simple variables, constants, pointers. Their size does not depend on  $n$ .
- **Dynamic memory** – arrays and tables whose size depends on  $n$  (e.g. `malloc(n * sizeof(int))`).
- **Recursion stack** – every active call keeps its own frame with parameters, locals, and a return address.

## Space at a glance

---

| Growth      | Simplest pattern                                  | Complexity |
|-------------|---------------------------------------------------|------------|
| Constant    | Fixed number of variables, independent of $n$     | 1          |
| Logarithmic | Recursion that halves the input (depth $\log n$ ) | $\log n$   |
| Linear      | Array of $n$ elements, or recursion depth $n$     | $n$        |
| Quadratic   | An $n \times n$ matrix                            | $n^2$      |
| Cubic       | An $n \times n \times n$ table                    | $n^3$      |

# 14, 15

---

```
1  int sumArray(int arr[], int n) {  
2      int s = 0;  
3      for (int i = 0; i < n; i++) {  
4          s += arr[i];  
5      }  
6      return s;  
7  }
```

```
1  int* copyArray(int arr[], int n) {  
2      int *b = malloc(n * sizeof(int));  
3      for (int i = 0; i < n; i++) {  
4          b[i] = arr[i];  
5      }  
6      return b;  
7  }
```

# 16, 17

---

```
1 void makeGrid(int n) {  
2     int grid[n][n];  
3     for (int i = 0; i < n; i++)  
4         for (int j = 0; j < n; j++)  
5             grid[i][j] = i * j;  
6 }
```

```
1 void reverse(int arr[], int n) {  
2     for (int i = 0; i < n / 2; i++) {  
3         int tmp = arr[i];  
4         arr[i] = arr[n - 1 - i];  
5         arr[n - 1 - i] = tmp;  
6     }  
7 }
```

# 18, 19

---

```
1  int factorial(int n) {  
2      if (n <= 1) return 1;  
3      return n * factorial(n - 1);  
4  }
```

```
1  int binarySearchRec(int arr[], int low, int high, int x) {  
2      if (low > high) return -1;  
3      int mid = low + (high - low) / 2;  
4      if (arr[mid] == x) return mid;  
5      if (arr[mid] < x) return binarySearchRec(arr, mid + 1, high, x);  
6      return binarySearchRec(arr, low, mid - 1, x);  
7  }
```

# Recursion and the Call Stack

---

Each active call keeps its own frame on the stack, so **space = maximum recursion depth**, not the total number of calls.

- `factorial(n)`: depth  $n \Rightarrow O(n)$
- `binarySearchRec`: depth  $\log n \Rightarrow O(\log n)$
- Example 7:  $2^n$  calls, but depth only  $n \Rightarrow O(n)$

## Revisiting Examples 1–13

---

| Example     | Where the memory goes                                                      | Space    |
|-------------|----------------------------------------------------------------------------|----------|
| 1–6, 10, 11 | A few variables ( <i>i</i> , <i>j</i> , <i>k</i> , <i>x</i> , <i>out</i> ) | 1        |
| 7, 8        | Recursion depth <i>n</i> (calls run one after another)                     | <i>n</i> |
| 9           | Recursion depth <i>n</i> (loop calls are sequential)                       | <i>n</i> |
| 12          | The result matrix, $n \times n$ for square inputs                          | $n^2$    |
| 13          | Only low, high, mid                                                        | 1        |



4

Exam Practice

# Exam Practice I

Analyze the time and space complexity of the following function:

```
1 void analyze(int arr[], int n)
2 {
3     int sum = 0;
4
5     // Part 1
6     for (int i = 0; i < n; i++)
7         sum += arr[i];
8
9     // Part 2
10    for (int i = 0; i < n; i++) {
11        int j = 1;
12
13        while (j < n)
14            j *= 2;
15
16        for (int k = 0; k < n; k++)
17            printf("%d ", arr[i]);
18    }
19
20    // Part 3
21    int *temp = malloc(n * sizeof(int));
22
23    for (int i = 0; i < n; i++)
24        temp[i] = arr[i];
25}
```

# Exam Practice II

---

```
26 // Part 4
27 for (int i = 0; i < n; i++) {
28     for (int j = i + 1; j < n; j++) {
29         if (arr[i] > arr[j]) {
30             int x = arr[i];
31             arr[i] = arr[j];
32             arr[j] = x;
33         }
34     }
35 }
36
37 free(temp);
38 }
```

# Key Takeaways

---

## Time Complexity

- Count how operations **grow with**  $n$ , not exact totals
- Loops  $\rightarrow$  linear; nested loops  $\rightarrow$  multiply
- Halving each step (e.g.  $i \neq 2$ )  $\rightarrow \log n$
- Recursion: branches  $\times$  depth  $\rightarrow$  total calls

## Space Complexity

- Auxiliary space: variables, arrays, **recursion stack**
- Fixed variables  $\rightarrow O(1)$ , regardless of loops
- An array/table sized by  $n \rightarrow O(n)$ ,  $O(n^2)$ , ...
- Recursion space = **max depth**, not number of calls

---

**Big-O (worst case) · Big- $\Omega$  (best case) · Big- $\Theta$  (tight bound)**

---

# Thank you

Questions welcome